Predicting NBA Game Outcomes and Identifying Mispriced Contracts in Sports Prediction Markets Using Machine Learning

Noah Arooji (nun2as)
Austin Vu (tcu3wh)
Azim Abdulmajeeth (zwf8qy)
Ciaran Jones (gff7en)

Abstract

This progress report is a follow up to our original proposal on NBA game prediction using the nba_api and sports market contracts. So far, we have completed the prediction market contract data collection and preprocessing pipeline. This pipeline merges NBA team features with Kalshi market data, we have also implemented an initial logistic regression baseline. On the held-out test set, this baseline achieves 71.58% accuracy with probability-based metrics, which shows a strong foundation for remaining model implementations as well as testing after.

1 Problem Statement

Sports prediction markets allow users to trade contracts on the outcomes of real-world events, including NBA basketball games. These platforms have operations resembling financial markets and behave similarly. Each contract is priced between $0 and $1, and the price reflects the market's implied probability of an outcome occurring. For example, a contract priced at $0.65 for a team to win implies the market believes that team has a 65% chance of winning. As these markets grow in volume and popularity, they create an opportunity to apply machine learning to detect mispriced contracts, meaning cases where a model trained on years of historical information has a estimated probability which differs meaningfully from current market probability.

Traditional sports betting often relies on expert intuition or simple statistical models. While sportsbooks use advanced internal models to set efficient lines, prediction market prices are set by participants and may adjust more slowly to new information such as matchup-specific factors. This project aims to take advantage of potential discrepancies by training supervised learning models to predict the true probability of NBA game outcomes, including winners and over/under totals. These predictions will then be compared to contract prices on Kalshi and Polymarket to find any possible discrepancies.

The model inputs will include features based on team and individual statistics, and current market prices. The output will be a predicted probability for each game outcome, so that we can flag probabilities that vary significantly from the market's implied probability.

2 Literature Review

The application of machine learning in sports outcome prediction is a well defined task. Researchers have applied logistic regression, support vector machines, and neural networks to predict outcomes in the NBA with varying degrees of success, generally achieving classification accuracies in the range of 65–72% depending on feature engineering and model complexity [1, 2]. A common finding across the literature is that ensemble methods, particularly gradient-boosted decision trees, will outperform individual classifiers in this domain because of their ability to capture interactions between features such as pace, rest days, and home-court advantage [3].

Prediction markets have also received significant attention in the economics and finance fields. Some studies show that prediction market prices are generally well-calibrated, so a contract priced at $0.70 wins approximately 70% of the time [4]. Researchers have also identified systematic inefficiencies, including the favorite-longshot bias and slow adjustment to breaking information, which create opportunities for informed and rapid decision making [5]. Recent work has attempted to combine machine learning predictions with market prices to generate profitable trading strategies in both sports and political prediction markets [6].

This project continues to draw on influences from both areas by using classification models from our machine learning curriculum to generate probability estimates. The probabilities are then evaluated on predictive accuracy as well as simulated profitability when used to trade against prediction market prices.

3 Dataset

All data for this project is collected from free, publicly available sources. Team-level NBA statistics are sourced from the `nba_api` Python package, which provides access to NBA.com game logs and advanced team statistics. Historical and real-time prediction market contract prices are collected from the Kalshi API, which serves as our primary source for market-implied probabilities in both training and later evaluation. We may also use The Odds API to pull current-day sportsbook lines if we test the models on live games.

So far, we have successfully implemented the moneyline portion of our data pipeline. From `nba_api`, we fetch both base and advanced team game logs to engineer pregame features. These features include rest days since the previous game, rolling 5-game and 10-game win percentage, rolling offensive rating, rolling defensive rating, rolling pace, and season-to-date home/away split win percentage. Crucially, these features are calculated chronologically and shifted so that each row only uses information available before the game being predicted, preventing leakage from future or same-game information.

We then convert the team-level data into a game-level moneyline dataset where each row corresponds to one NBA game, containing home and away feature values, metadata (date, team abbreviations), and a binary target variable indicating a home team win. On the

Kalshi side, our pipeline accesses both historical and live market endpoints, matching moneyline markets to NBA games using game dates and team identities. This has yielded a merged moneyline dataset containing 912 matched games with both NBA features and a Kalshi home-win price.

At this stage, the moneyline data collection and merging pipeline is fully functioning, while the over/under version of the dataset is slated for our next phase of development.

4 Algorithms and Models

We are implementing several supervised learning models covered in the machine learning curriculum, selecting those best suited for structured tabular prediction. At this stage, we have established our evaluation framework and completed our baseline model, with more complex ensemble methods planned for the immediate next steps.

4.1 Logistic Regression (Baseline Completed)

Logistic regression serves as a baseline architecture for this binary classification task. It is highly interpretable, allowing us to inspect the learned coefficients and understand which features contribute to the predicted outcome with the largest accuracy. We chose to use L2 regularization to prevent overfitting given the moderate number of features relative to the number of games per season.

4.1.1 Architecture. Logistic regression models the home-win probability as:

$$\hat{p}_k = \sigma(\mathbf{w}^\top \tilde{\mathbf{x}}_k + b) = \frac{1}{1 + \exp(-(\mathbf{w}^\top \tilde{\mathbf{x}}_k + b))} \tag{1}$$

where $\mathbf{w} \in \mathbb{R}^d$ is the weight vector, $b \in \mathbb{R}$ is the bias term, and $\sigma(\cdot)$ denotes the sigmoid function. This model serves as a linear baseline, providing interpretable coefficients that quantify the marginal contribution of each feature to the predicted log-odds of a home win.

Currently, the logistic regression baseline is fully implemented for the moneyline task. The model is trained on 14 standardized basketball-derived numeric inputs: home and away rest days, season-to-date split win percentages, rolling 5-game and 10-game win rates, and rolling 5-game offensive rating, defensive rating, and pace. To ensure a temporally mindful evaluation that reflects real-world forecasting, we use a chronological split. The earliest 70% of games in our merged dataset are used for training, while the most recent 20% are held out for testing and the final 10% as a validation set. We intentionally exclude metadata and Kalshi price columns from the feature matrix so the baseline learns purely from basketball statistics rather than the market's implied probabilities.

4.2 Ensemble Methods (In Progress)

With the baseline model established, our next step is to implement ensemble methods, specifically Random Forests and Gradient-Boosted Decision Trees (e.g., XGBoost). We expect these models to deliver the strongest predictive performance due to their ability to capture more fine grained interactions.

4.2.1 Architecture. A Random Forest constructs an ensemble of B decision trees $\{T_1, T_2, \ldots, T_B\}$, each trained on a bootstrap sample of the training data with a random subset of features considered at each split. The predicted probability for game g_k is the average of the individual tree predictions:

$$\hat{p}_k = \frac{1}{B} \sum_{b=1}^{B} T_b(\tilde{\mathbf{x}}_k) \tag{2}$$

where $T_b(\tilde{\mathbf{x}}_k) \in [0, 1]$ is the fraction of training samples in the leaf node reached by $\tilde{\mathbf{x}}_k$ in tree b that belong to the positive class.

Random Forest models themselves are robust to overfitting and naturally handle feature interactions without extensive feature engineering. Gradient boosting builds trees sequentially to correct prior errors and has been shown to excel in structured tabular prediction tasks. These models also provide feature importance scores, which will help identify whether team performance metrics or scheduling factors carry the most predictive weight.

These models are something that we are still working on and will be evaluated later this week using the exact same 14-feature dataset and chronological split established by our logistic regression baseline. We will tune key hyperparameters such as the number of estimators, maximum tree depth, and learning rate to optimize both classification accuracy and the calibration of the predicted probabilities using our validation set.

5 Evaluation

Model evaluation for the full project is conducted along two dimensions: predictive quality and economic value. For predictive quality, we use accuracy (overall classification correctness), ROC AUC, log-loss (which penalizes confident but incorrect predictions), and Brier score (the mean squared error of predicted probabilities against binary outcomes). Log-loss and Brier score are critical because identifying mispriced contracts requires well-calibrated probability estimates rather than simple binary classifications.

5.1 Predictive Quality Metrics

We evaluate each model using four complementary metrics on the test set $\mathcal{G}_{\text{test}}$:

- **Accuracy**: $\text{Acc} = \frac{1}{|\mathcal{G}_{\text{test}}|} \sum_{g_k \in \mathcal{G}_{\text{test}}} \mathbf{1}[\hat{y}_k = y_k]$
- **ROC AUC**: the area under the receiver operating characteristic curve, measuring ranking quality.
- **Log-loss**: $\mathcal{L}_{\log} = -\frac{1}{|\mathcal{G}_{\text{test}}|} \sum_{g_k \in \mathcal{G}_{\text{test}}} \left[y_k \log \hat{p}_k + (1-y_k) \log(1-\hat{p}_k) \right]$
- **Brier score**: $\text{BS} = \frac{1}{|\mathcal{G}_{\text{test}}|} \sum_{g_k \in \mathcal{G}_{\text{test}}} (\hat{p}_k - y_k)^2$

Log-loss and Brier score are particularly critical because identifying mispriced contracts requires well-calibrated probability estimates rather than simple binary classifications.

At this progress-report stage, we have evaluated our completed logistic regression baseline on predictive quality. Using our chronological split, the model was trained on 729 games and evaluated on a held-out test set of 183 later games. Table 1 summarizes these initial results.

These results are promising for an initial baseline. The model achieves 71.58% accuracy on held-out games, which aligns with the higher end of the range cited in our literature review. The ROC AUC of 0.7865 suggests effective separation of home wins from

Table 1: Initial logistic regression moneyline results on the held-out chronological test split.

Metric	Value
Training rows	729
Test rows	183
Number of features	14
Accuracy	0.7158
ROC AUC	0.7865
Log-loss	0.5689
Brier score	0.1919

away wins, while the log-loss and Brier score indicate reasonably calibrated outputs.

For the second dimension of evaluation, economic value, we will conduct a simulated trading test. The strategy will trigger a simulated trade when the model's predicted probability diverges from the Kalshi implied probability by more than a specified threshold, allowing us to measure the return on investment (ROI). We are currently refining the Kalshi backtesting pipeline to ensure the market probabilities used in simulation correspond to reliable pregame prices. Once finalized, we will compare our upcoming ensemble models against our baselines, which include a naive home-team predictor, a market-implied probability baseline, and this current logistic regression model.

6 Timeline

The project is progressing well along our planned seven-week schedule. During the first few weeks, we successfully completed data collection from the `nba_api` and Kalshi API, constructed our chronologically shifted feature set, and implemented and evaluated our initial logistic regression baseline for the moneyline task.

Over the next two weeks, we will finish implementing and training our core ensemble models (Random Forests and Gradient-Boosted Decision Trees) for the moneyline task, while simultaneously expanding our dataset pipeline to include over/under totals. Following this, we will dedicate a week to the simulated trading test, where we will fully implement the backtesting pipeline to compare our models against Kalshi's implied probabilities and measure ROI across different trading thresholds. The final week will be focused entirely on preparing the final project report and presentation materials.

7 Expected Outcome

We expect that ensemble methods, especially gradient-boosted decision trees, to have the strongest prediction accuracy and best-calibrated probabilities, consistent with our findings in existing literature on tabular prediction tasks. Logistic regression should serve as a competitive and interpretable baseline, and the comparison between these two model families will help characterize whether the nonlinear feature interactions captured by tree-based ensembles provide meaningful lift over a linear model in our specific setting. We anticipate overall classification accuracy in the range of 65–70%, which is consistent with prior work on NBA game prediction.

More importantly, we expect to identify a subset of games where the model's predicted probability diverges meaningfully from the Kalshi contract price, and we hypothesize that a simple threshold-based trading strategy will yield positive simulated ROI on these contracts. Even if the absolute accuracy improvement over the market is modest, the project will demonstrate the full pipeline of training a probabilistic classifier, calibrating its outputs, and applying it to a real-world decision-making context in prediction markets.

References

[1] F. Thabtah, L. Zhang, and N. Abdelhamid. 2019. NBA Game Result Prediction Using Feature Analysis and Machine Learning. *Annals of Data Science*, 6(1), 103–116.

[2] B. Loeffelholz, E. Bednar, and K. Bauer. 2009. Predicting NBA Games Using Neural Networks. *Journal of Quantitative Analysis in Sports*, 5(1).

[3] T. Chen and C. Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.

[4] J. Wolfers and E. Zitzewitz. 2004. Prediction Markets. *Journal of Economic Perspectives*, 18(2), 107–126.

[5] E. Snowberg and J. Wolfers. 2010. Explaining the Favorite-Long Shot Bias: Is it Risk-Love or Misperceptions? *Journal of Political Economy*, 118(4), 723–746.

[6] R. Beal, S. Middleton, T. Norman, and S. Ramchurn. 2021. Combining Machine Learning and Human Experts to Predict Match Outcomes in Football: A Baseline Model. In *Proceedings of the 35th AAAI Conference on Artificial Intelligence*, 15447–15451.